

Санкт-Петербургский государственный университет

Кафедра, информационно-аналитических систем

Группа 24.Б42-мм

Реализация алгоритма GA-RFD в “Desbordante”

Виноградов Дмитрий Валентинович

Отчёт по учебной практике
в форме «Решение»

Научный руководитель:
асс. кафедры ИАС Г. А. Чернышев

Санкт-Петербург
2026

Оглавление

Введение	3
1. Постановка задачи	5
2. Обзор	6
2.1. Функциональные зависимости и их обобщения	6
2.2. Аналогии	8
3. Предварительные сведения	10
3.1. Основные понятия	10
3.2. Поиск RFD	11
4. Реализация	19
4.1. Архитектура реализации примитива	19
4.2. Интеграция с Python	21
4.3. Пример использования	23
5. Эксперимент	25
5.1. Исследовательские вопросы	25
5.2. Условия эксперимента	25
5.3. RQ1. Производительность алгоритма	26
5.4. RQ2. Обнаружение релаксированных зависимостей . . .	27
5.5. RQ3. Влияние порога сходства	29
5.6. Итоговые выводы	29
Заключение	31
Список литературы	32

Введение

Анализ данных с целью получения метаданных называют [2] профилированием. К метаданным относятся характеристики набора данных, статистики атрибутов и различные виды закономерностей между ними. Полученная информация используется для решения практических задач, таких как очистка данных, обнаружение аномалий, дедупликация, интеграция данных и повышение качества аналитики.

Одним из современных инструментов для профилирования данных является платформа Desbordante [3], предоставляющая реализации алгоритмов поиска зависимостей и статистик. Платформа ориентирована на решение прикладных задач и активно используется в исследованиях и проектах в области науки о данных.

Одно из направлений профилирования — выявление закономерностей. Конкретный вид таких закономерностей определяется природой данных: они могут быть табличными, графовыми, транзакционными и т.д. В настоящей работе рассматриваются исключительно табличные данные.

Примером закономерности в таблицах служат функциональные зависимости (Functional Dependencies, FD). Их можно применять для нормализации данных, однако для задач дедупликации и очистки они подходят не всегда: в реальных данных значения атрибутов у записей, описывающих один и тот же объект, могут различаться из-за опечаток, различных форматов или ошибок ввода.

Для таких задач лучше подходят сопоставляющие зависимости (Matching Dependencies, MD) [5], которые позволяют сопоставлять записи на основе схожести, а не строгого равенства. Наличие приближённых функциональных зависимостей (Approximate FD, AFD) [7] с малой ошибкой даёт возможность выявлять аномалии, очищать данные и находить близкие к выполнению, но не абсолютные закономерности, что полезно для аналитических задач.

Наиболее гибким инструментом, сочетающим устойчивость к опечаткам и к исключениям, являются ослабленные функциональные за-

зависимости (Relaxed FD, RFD) [1]. Заметим что, в отличие от обзорных работ, где термин RFD используется в широком смысле для обозначения различных обобщений FD, в данной работе под RFD понимается конкретный тип зависимостей, предложенный в работе [1].

RFD утверждает: «Если значения на атрибутах X достаточно похожи (в смысле Φ_1), то с вероятностью не менее ϵ значения на атрибутах Y также похожи (в смысле Φ_2)».

Такая трактовка позволяет обнаруживать закономерности в реальных «грязных» данных, содержащих опечатки, вариации и исключения, что делает RFD перспективным инструментом для задач профилирования и очистки данных.

Для поиска RFD в работе [1] был предложен генетический алгоритм GA-RFD. В платформе Desbordante [3] уже реализованы алгоритмы поиска приближённых функциональных зависимостей [14] и сопоставляющих зависимостей [15]. В настоящей работе выполняется реализация алгоритма GA-RFD на языке C++ и его интеграция в платформу, что расширяет её возможности по поиску закономерностей в табличных данных.

1. Постановка задачи

Настоящая работа выполняется в контексте продолжающегося развития проекта Desbordante. Основная цель — реализация алгоритма GA-RFD на C++ в составе платформы Desbordante с обеспечением доступа к нему через Python-интерфейс.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Реализовать алгоритм GA-RFD в ядре Desbordante;
2. Создать тесты для проверки корректности работы алгоритма;
3. Обеспечить доступ к алгоритму из Python с помощью библиотеки `pybind11`;
4. Провести сравнительный эксперимент, оценивающий производительность и качество найденных RFD, сопоставив результаты с существующими алгоритмами поиска FD и AFD.

2. Обзор

2.1. Функциональные зависимости и их обобщения

Функциональная зависимость (FD) — это правило, утверждающее, что если два кортежа совпадают по набору атрибутов X , то они обязаны совпадать и по набору атрибутов Y . Функциональные зависимости были введены в рамках реляционной модели данных и являются одним из фундаментальных понятий теории баз данных. Они используются для нормализации схем и выявления избыточности данных.

Для поиска FD предложено множество алгоритмов. Классическим алгоритмом является TANE [13], который выполняет поиск в решётке множеств атрибутов и использует разбиения кортежей для эффективной проверки зависимостей.

Современные алгоритмы, такие как НуFD [12], комбинируют выборку данных и точную проверку кандидатов, что позволяет масштабироваться на наборы данных с миллионами записей и десятками атрибутов.

Алгоритм Руго [8] развивает идеи поиска зависимостей за счёт использования стратегии *separate-and-conquer* и выборочного анализа данных. В отличие от классических методов, он ориентирован на обнаружение не только точных, но и приближённых зависимостей, что позволяет эффективно работать с данными, содержащими шум и ошибки.

Ещё одним современным подходом является FDHits [4], который сводит задачу поиска функциональных зависимостей к задаче перечисления минимальных hitting set'ов. Использование эффективных методов перечисления и гибридной проверки кандидатов обеспечивает высокую производительность и низкое потребление памяти по сравнению с рядом существующих алгоритмов.

Приближённая функциональная зависимость (AFD) ослабляет FD по полноте выполнения: правило должно выполняться не для всех, а для *большинства* кортежей. В работе TANE [13] была показана возможность эффективного обнаружения приближённых функциональных зависимостей за счёт использования разбиений кортежей. Доля нару-

шений измеряется с помощью специальных метрик [9] (например, g_1), и пользователь задаёт максимально допустимый уровень ошибки. AFD позволяют выявлять закономерности, которые нарушаются в небольшом проценте случаев, что полезно для обнаружения аномалий и очистки данных. Однако AFD по-прежнему используют строгое равенство значений, поэтому они не устойчивы к опечаткам и вариациям в данных.

Сопоставляющая зависимость (MD) решает проблему опечаток, заменяя равенство на *схожесть* значений. MD утверждает, что если значения на атрибутах X похожи (согласно заданной метрике и порогу), то значения на атрибутах Y также должны быть похожи. Поиск сопоставляющих зависимостей существенно сложнее поиска FD, поскольку пространство поиска включает не только множества атрибутов, но и параметры функций сходства. Одним из современных алгоритмов является НуMD [5], использующий гибридный подход, аналогичный НуFD: кандидаты генерируются по выборке данных, после чего выполняется их точная проверка.

Наиболее гибким инструментом, сочетающим устойчивость к опечаткам и к исключениям, являются ослабленные функциональные зависимости (RFD). В отличие от обзорных работ, где термин RFD используется в широком смысле для обозначения различных обобщений FD, в данной работе под RFD понимается конкретный тип зависимостей, предложенный в [1]. RFD одновременно ослабляют FD по двум направлениям:

- *По сравнению значений*: вместо равенства используются метрики схожести с заданными порогами (границами решения);
- *По полноте выполнения*: правило должно выполняться не для всех, а для доли пар кортежей, не меньшей заданного порога ϵ .

Таким образом, RFD позволяют формулировать правила вида: «Если два кортежа достаточно похожи по X , то с вероятностью не менее ϵ они будут похожи и по Y ». Это делает RFD наиболее подходящим

инструментом для анализа реальных «грязных» данных, содержащих одновременно опечатки, вариации и редкие исключения.

В таблице 1 приведено сравнение описанных типов зависимостей по ключевым характеристикам.

Таблица 1: Сравнение FD, AFD, MD и RFD

Property	FD	AFD	MD	RFD
Ослабление	Нет	По объёму (исключения)	По значению (схожесть)	По объёму и по значению
Функция сравнения	Равенство	Равенство + мера ошибки	Пороговая схожесть	Пороговая схожесть
Устойчивость к зашумлён- ным данным	Нет	К исключениям	К опечаткам	К опечаткам и исключениям
Уникальная особенность	Строгое равенство	«Почти всегда равно»	Сопоставление записей по схожести	Двойная релаксация

Из таблицы 1 видно, что различия между типами зависимостей определяются двумя аспектами: способом сравнения значений и допустимостью нарушений. FD требуют строгого равенства и полного выполнения. AFD допускают нарушения, но сохраняют равенство. MD заменяют равенство схожестью, но не допускают исключений. RFD сочетают оба подхода — используют метрики схожести и допускают нарушения. Благодаря двойной релаксации RFD являются наиболее гибким инструментом для анализа реальных данных с опечатками и аномалиями.

2.2. Аналоги

Среди существующих инструментов профилирования данных:

- Pandas и Pandas Profiling [10,11] предоставляют описательные статистики, но не реализуют поиск RFD;

- Metanome [6] содержит алгоритмы для поиска некоторых типов зависимостей, однако поиск RFD в том виде, как он определён в [1], в нём отсутствует.

Таким образом, реализация GA-RFD в Desbordante заполняет важную нишу — поиск правил, устойчивых одновременно к опечаткам (схожесть) и к исключениям (покрытие).

3. Предварительные сведения

В работе [5], посвящённой реализации алгоритма НуMD, вводятся следующие определения, которые полезны для понимания различия и сходства между подходами.

Граница решения (decision boundary) — это пороговое значение метрики схожести, определяющее, считаются ли два значения достаточно похожими. Если результат применения метрики к паре значений не меньше границы решения, то значения считаются схожими, в противном случае — несхожими.

Естественная граница решения (natural decision boundary) — это такая граница решения, которая равна значению схожести, реально встречающемуся в данных между некоторой парой значений атрибутов. В алгоритме НуMD использование естественных границ решения позволяет сузить пространство поиска, рассматривая только те пороги, которые действительно имеют смысл для конкретного набора данных.

В отличие от НуMD, где границы решения извлекаются из данных (естественные границы), в GA-RFD используются задаваемые пользователем фиксированные пороги схожести. Это упрощает настройку алгоритма и делает его более гибким для пользователя.

3.1. Основные понятия

В данном разделе используются определения и обозначения, введённые в работе [1], в которой был предложен рассматриваемый в настоящей работе тип ослабленных функциональных зависимостей.

Ослабленная функциональная зависимость (RFD) обобщает классическую FD по двум направлениям: по способу сравнения значений и по полноте выполнения.

Метрика схожести — функция, ставящая в соответствие двум значениям вещественное число из отрезка $[0, 1]$, где 1 означает полную идентичность. Значения считаются схожими, если результат применения метрики к ним не меньше заданного порога (границы решения).

Хотя в работе [1] используются другие обозначения порогов схожести, далее будут применяться понятия NYMD из работы [5], поскольку они являются более распространёнными в литературе и удобны для описания алгоритмов поиска зависимостей на основе схожести.

Ослабленная функциональная зависимость для одной таблицы записывается в виде

$$\mathbb{D}_c : X_{\Phi_1} \xrightarrow{\Psi \geq \epsilon} Y_{\Phi_2},$$

где:

- X, Y — множества атрибутов (левая и правая части);
- Φ_1, Φ_2 — условия схожести (метрика и порог);
- Ψ — мера покрытия (confidence);
- ϵ — минимальная доля пар кортежей, для которых правило должно выполняться;
- $\mathbb{D}_c$ — подмножество кортежей, отфильтрованное предикатами c . В текущей реализации фильтрация не применяется, поэтому $\mathbb{D}_c$ фактически совпадает со всем множеством кортежей.

На практике RFD интерпретируется так: «Если два кортежа достаточно похожи по X , то с вероятностью не менее ϵ они будут похожи и по Y ».

3.2. Поиск RFD

Пространство поиска RFD значительно больше, чем для FD, поскольку для каждого атрибута может быть много возможных границ решения. Поэтому полный перебор нерационален.

Важной особенностью GA-RFD является его приближенный характер: алгоритм не гарантирует обнаружение всех существующих RFD (возможны пропуски), однако не выдаёт ложных срабатываний — все найденные зависимости действительно выполняются на данных с заданной уверенностью. Это означает, что пост-валидация результатов

не требуется, что существенно упрощает практическое использование алгоритма.

В работе [1] предложен генетический алгоритм GA-RFD, который использует следующие понятия:

Разностная таблица — бинарная матрица, где строки соответствуют парам кортежей, столбцы — атрибутам, а значение равно 1, если пара похожа по данному атрибуту, и 0 в противном случае.

Поддержка (support) — доля пар кортежей, похожих по заданному набору атрибутов. Формально: $\text{supp}(X) = \frac{|\{(t_1, t_2): t_1 \approx_X t_2\}|}{N}$, где N — общее число пар кортежей.

Уверенность (confidence) — доля пар, похожих по правой части, среди пар, похожих по левой части. Формально: $\text{conf}(X \rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)}$. Это значение выступает в роли фитнес-функции: чем оно выше, тем лучше кандидат.

В качестве примера, рассмотрим набор данных о мерах тела, взятый из оригинальной статьи [1] (табл. 3). Для каждой пары кортежей будем считать значения схожими, если выполняются следующие условия:

Рост: $|h_1 - h_2| \leq 2$;

Вес: $|w_1 - w_2| \leq 5$;

Размер обуви: $|s_1 - s_2| \leq 1$.

Указанные условия задаются с помощью метрики абсолютной разности (`abs_threshold_metric(diff)`, где `diff` — порог), которая сравнивает значения атрибута с заданным порогом. Метрика возвращает значение 1, 0, если абсолютная разность двух значений не превышает порог, иначе 0, 0. В рассматриваемом примере для атрибута «Рост» используется порог 2, для атрибута «Вес» — порог 5, а для атрибута «Размер обуви» — порог 1.

Разностная таблица (табл. 4) строится следующим образом: для каждой пары кортежей и для каждого атрибута вычисляется значение заданной пользователем метрики схожести. Если полученное значение не меньше порога (в данном случае порог равен 1.0, так как функция

схожести абсолютная разница, которая возвращает 1.0 или 0.0, если неравенство выполняется или нет соответственно), заданного для данного атрибута, то в соответствующую ячейку записывается 1, в противном случае — 0.

Таблица 3: Исходные данные

ID	Height	Weight	Shoe Size
1	175	70	40
2	175	75	39
3	175	69	40
4	176	71	40
5	178	81	41
6	169	73	37
7	170	62	39

Таблица 4: Разностная таблица

Pair ID	Height	Weight	Shoe Size
(1,2)	1	1	1
(1,3)	1	1	1
(1,4)	1	1	1
(1,5)	0	0	1
(1,6)	0	1	0
(1,7)	0	1	1
(2,3)	1	1	1
(2,4)	1	1	1
(2,5)	0	1	0
(2,6)	0	1	0
(2,7)	0	0	1
(3,4)	1	1	1
(3,5)	0	0	1
(3,6)	0	1	0
(3,7)	0	1	1
(4,5)	0	1	1
(4,6)	0	1	0
(4,7)	0	1	1
(5,6)	0	1	0
(5,7)	0	0	0
(6,7)	1	0	0

Рассмотрим возможную зависимость:

$$\text{ShoeSize}_{\text{abs_threshold}(1)} \rightarrow \text{Height}_{\text{abs_threshold}(2)}.$$

Поддержка (support) для атрибута Shoe Size:

$$\text{supp}(\text{ShoeSize}) = \frac{\text{число пар, похожих по размеру обуви}}{\text{общее число пар}} = \frac{13}{21}.$$

Поддержка для комбинации {ShoeSize, Height}:

$$\begin{aligned}\text{supp}(\text{ShoeSize}, \text{Height}) &= \frac{\text{число пар, похожих по обоим атрибутам}}{\text{общее число пар}} = \\ &= \frac{6}{21}.\end{aligned}$$

Тогда уверенность (confidence) зависимости:

$$\text{conf}(\text{ShoeSize} \rightarrow \text{Height}) = \frac{\text{supp}(\text{ShoeSize}, \text{Height})}{\text{supp}(\text{ShoeSize})} = \frac{6/21}{13/21} = \frac{6}{13} \approx 0.46.$$

При пороге $\epsilon = 0.9$ данная зависимость не является допустимой, поскольку $0.46 < 0.9$.

Цель генетического алгоритма GA-RFD — найти такие наборы атрибутов и пороги, чтобы уверенность была не ниже заданного значения ϵ .

Для решения этой задачи необходимо определить способ представления потенциальных RFD в виде, удобном для применения генетических операторов (скрещивания, мутации и отбора). Поэтому первым шагом является выбор схемы кодирования особей, описывающих кандидатов на роль искомым зависимостей.

Кодирование особей

Для применения генетического алгоритма к задаче поиска RFD необходимо определить способ представления кандидатов (особей) в виде, пригодном для выполнения генетических операторов. Следуя подходу работы [1], каждая особь кодирует множество атрибутов левой части зависимости (LHS) и один атрибут правой части (RHS). В данной реализации для этого используется битовая маска (bitmask): каждый бит 32-битного целого числа соответствует одному атрибуту и принимает значение 1, если атрибут входит в левую часть зависимости. Правая часть хранится отдельно в виде индекса атрибута (0..31). Такое представление обеспечивает компактное хранение особей и позволяет эффективно реализовать генетические операторы: мутацию посредством изменения отдельного бита и кроссовер посредством комбинирования

битовых масок родителей с использованием побитовых операций.

Генетические операторы

После того как определено представление особей, необходимо описать механизмы, с помощью которых популяция эволюционирует к более качественным решениям. Эти механизмы включают инициализацию, отбор, скрещивание и мутацию.

1. **Инициализация (Initialize).** Создаётся начальная популяция случайных особей. В битовом представлении каждый бит левой части инициализируется случайным образом (0 или 1) с вероятностью, обеспечивающей равномерное распределение, а правая часть выбирается случайным индексом атрибута. При этом контролируется, чтобы левая часть не была пустой и чтобы правый атрибут не входил в левую часть.
2. **Отбор (Selection).** Особи выживают с вероятностью, пропорциональной их приспособленности (confidence). Особи, удовлетворяющие порогу ϵ , гарантированно попадают в следующее поколение. Вероятность отбора:

$$P(\text{выбора}) = \begin{cases} 1, & \text{если } \text{conf} \geq \epsilon, \\ \frac{\text{conf}}{\epsilon}, & \text{в противном случае,} \end{cases}$$

где ϵ — заданный порог уверенности;

3. **Скрещивание (Crossover).** С вероятностью p_1 (в экспериментах $p_1 = 0,85$) выполняется одноточечный кроссовер: две родительские особи обмениваются частями генов, начиная со случайно выбранной позиции, порождая потомков;
4. **Мутация (Mutation).** С вероятностью p_2 (в экспериментах $p_2 = 0,3$) случайный ген особи заменяется на другой случайный индекс атрибута. Это вносит разнообразие и позволяет исследовать новые области пространства поиска;

5. **Завершение (Termination).** Алгоритм останавливается, когда все особи популяции достигают уверенности не ниже ϵ либо когда достигнуто максимальное число поколений **maxgen**. На выход подаются все RFD с уверенностью $\text{conf} \geq \epsilon$.

Схема работы алгоритма

Алгоритм циклически выполняет этапы отбора, кроссовера и мутации, пока не будет выполнено одно из условий:

- все особи популяции достигли уверенности не ниже ϵ ;
- достигнуто максимальное число поколений **maxgen**.

На рисунке 1 представлена блок-схема алгоритма GA-RFD.

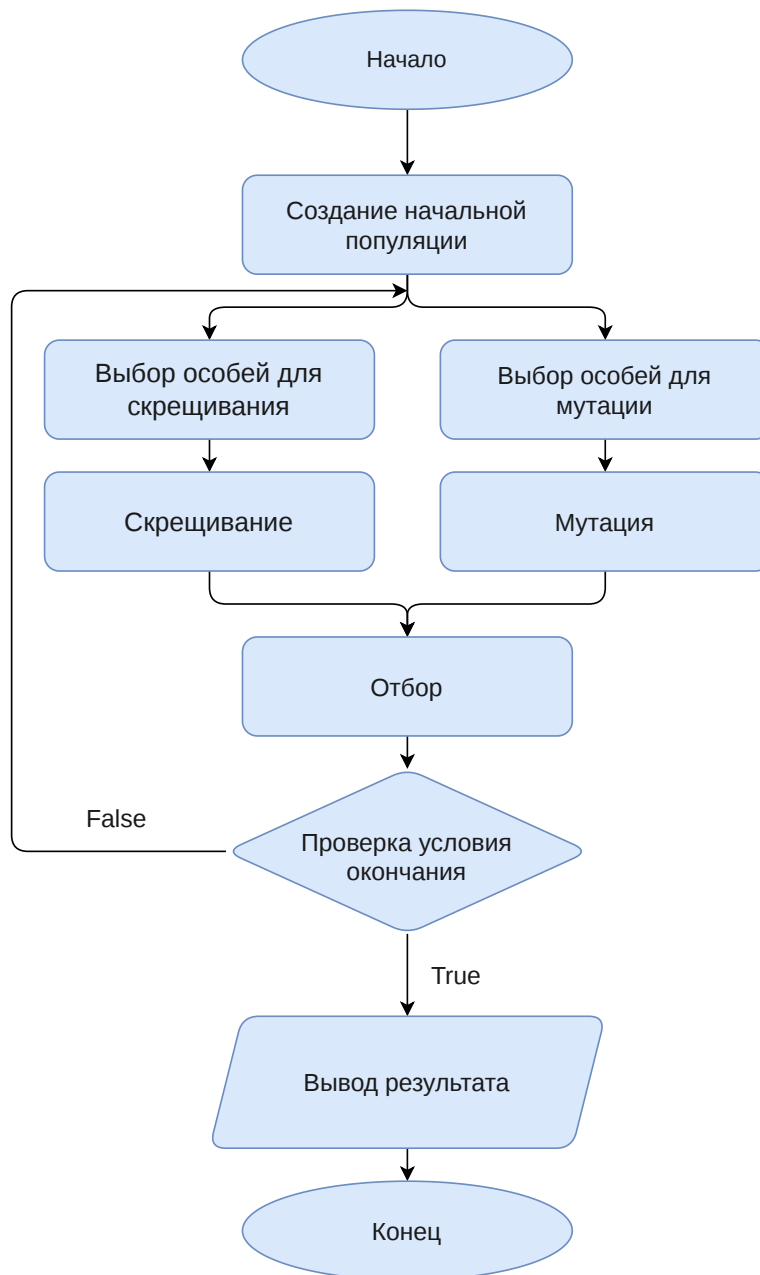


Рис. 1: Блок-схема алгоритма GA-RFD

Особенности хранения разностной таблицы

В реализации на C++ для хранения разностной таблицы применяются битовые маски, что значительно ускоряет вычисления поддержки. Для каждого атрибута хранится массив 64-битных целых чисел,

где каждый бит соответствует одной паре кортежей. Вычисление поддержки набора атрибутов сводится к побитовому “И” соответствующих масок, а подсчёт единичных битов выполняется аппаратной инструкцией `std::popcount`.

4. Реализация

4.1. Архитектура реализации примитива

Структура реализации алгоритма поиска релаксированных функциональных зависимостей представлена на рис. 2.

Базовым классом является `Algorithm`, входящий в состав платформы `Desbordante`. Он предоставляет общий интерфейс для всех алгоритмов профилирования данных, включая регистрацию параметров, загрузку входных данных, запуск вычислений и очистку внутреннего состояния. От него наследуется класс `Garfd`, реализующий генетический алгоритм поиска релаксированных функциональных зависимостей.

Загрузка данных выполняется в методе `LoadDataInternal`. Таблица считывается штатными средствами платформы через объект `InputTable`. После загрузки данные преобразуются во внутреннее представление `column_data_`, где каждый атрибут хранится как отдельный вектор значений. Такое представление упрощает поколонную обработку данных и позволяет использовать различные метрики сходства независимо для каждого атрибута.

Перед запуском генетического алгоритма выполняется построение разностной таблицы в методе `BuildSimilarityBitsets`. Для каждого атрибута формируется битовый вектор сходства `attr_similarity_bits_`. Каждый бит соответствует одной паре кортежей и принимает значение 1, если значения атрибута в данной паре считаются похожими согласно выбранной метрике и заданному порогу сходства. Использование битовых масок позволяет эффективно выполнять пересечение множеств похожих пар посредством побитовой операции `AND`.

Кандидат на релаксированную функциональную зависимость представляется структурой `Individual`. Левая часть зависимости кодируется битовой маской `lhs_mask`, а правая часть задаётся индексом атрибута `rhs_index`. Для каждой особи также хранятся значения поддержки и уверенности, вычисляемые функцией `Evaluate`.

Оценка особей основана на показателях поддержки и уверенности,

аналогичных используемым в задачах поиска ассоциативных правил. Для вычисления поддержки применяется метод `ComputeSupport`, который определяет количество пар кортежей, одновременно удовлетворяющих всем атрибутам рассматриваемого множества. Для сокращения числа повторных вычислений используется LRU-кэш `support_cache_`, сохраняющий ранее вычисленные значения поддержки.

Начальная популяция формируется методом `InitializePopulation`, который случайным образом генерирует допустимые комбинации левых и правых частей зависимостей. После этого запускается основной эволюционный цикл, включающий следующие этапы:

- вычисление приспособленности особей (`EvaluatePopulation`);
- вероятностный отбор (`Select`);
- рекомбинацию решений (`Crossover`);
- мутацию (`Mutate`);
- формирование нового поколения.

Функция приспособленности определяется через уверенность зависимости. Если уверенность особи превышает заданный пользователем порог ε , её приспособленность принимается равной единице. В противном случае используется отношение фактической уверенности к требуемому порогу.

Алгоритм завершает работу при достижении максимального числа поколений либо при выполнении критерия останова, когда все особи текущей популяции удовлетворяют минимальному уровню уверенности. На заключительном этапе метод `Finalize` формирует итоговое множество релаксированных функциональных зависимостей, отбрасывая решения с недостаточной уверенностью.

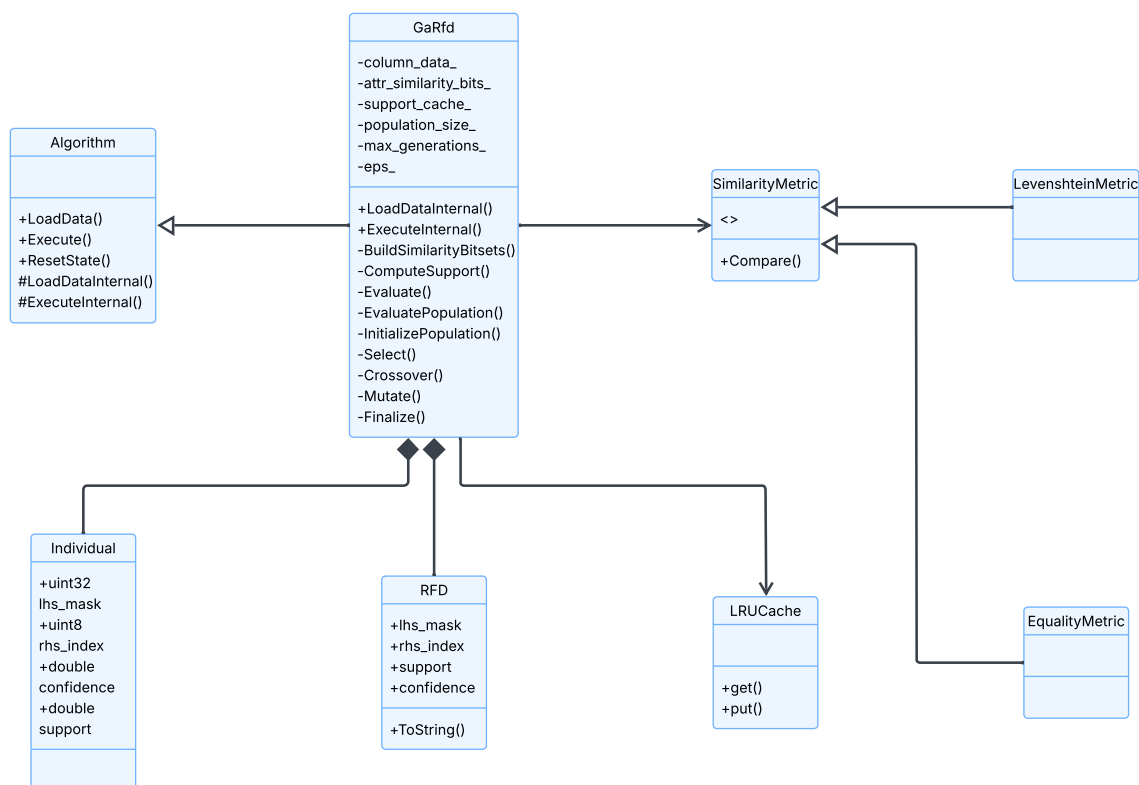


Рис. 2: Упрощённая UML-диаграмма классов GA-RFD

4.2. Интеграция с Python

Платформа Desbordante предоставляет интерфейс для языка Python, что позволяет использовать алгоритм GA-RFD в среде специалистов по данным без необходимости работы с C++. Интеграция выполнена с помощью библиотеки `pybind11`, которая автоматически генерирует Python-обёртки для C++ классов и методов. Пользователи могут загружать таблицы из объектов `pandas.DataFrame`, передавая их в метод `load_data`. Результаты возвращаются в виде списка объектов `RFD`, содержащих маску левой части, индекс правой части, уверенность и поддержку.

Параметры алгоритма

Основные параметры, управляющие работой GA-RFD, задаются через метод `set_option`:

population_size — число особей (по умолчанию 1024);
max_generations — число поколений (по умолчанию 32);
crossover_probability — вероятность скрещивания (по умолчанию 1.0);
mutation_probability — вероятность мутации (по умолчанию 0.1);
minconf — порог уверенности ϵ (по умолчанию 1.0);
min_similarity — порог(и) схожести для атрибутов;
seed — зерно для генератора случайных чисел;
cache_size — размер кэша поддержки (по умолчанию 10000).

Поскольку алгоритм использует случайность, для получения воспроизводимых результатов настоятельно рекомендуется фиксировать **seed**.

Метрики схожести

Метрики схожести определяют, насколько два значения считаются похожими. Они задаются через метод **set_metrics**, который принимает список функций (по одной на каждый атрибут). Автором были реализованы встроенные метрики:

- **abs_diff_metric()** — для числовых данных: $1 - \frac{|x-y|}{\max(|x|, |y|)}$. Подходит для величин с естественным нулём;
- **abs_threshold_metric(diff)** — для числовых данных: возвращает 1, если $|x - y| \leq diff$, и 0 в противном случае;
- **equality_metric()** — возвращает 1 при точном равенстве значений, иначе 0;
- **levenshtein_metric()** — для строк: $1 - \frac{\text{расстояние Левенштейна}}{\max(\text{len}(x), \text{len}(y))}$.

Пользователь также может передать собственную функцию схожести вида

$$f(a, b) \rightarrow \text{float},$$

определённую на Python.

4.3. Пример использования

Ниже приведён минимальный пример, демонстрирующий типичный рабочий процесс:

Листинг 1: Пример использования GA-RFD из Python

```
import desbordante

# Load data
algo = desbordante.rfd.algorithms.GaRfd()
algo.load_data(table=('data.csv', ',', True))

# Set metrics (absolute difference for all attributes)
abs_diff = desbordante.rfd.abs_diff_metric()
algo.set_metrics([abs_diff, abs_diff, abs_diff])

# Set options
algo.set_option("min_similarity", 0.95)
algo.set_option("minconf", 0.7)
algo.set_option("max_generations", 100)
algo.set_option("seed", 42)

# Run and get results
algo.execute()
rfd = algo.get_rfd()
for rfd in rfd:
    print(rfd)
```

Каждый объект RFD содержит поля `lhs_mask` (битовая маска левой части), `rhs_index` (индекс правого атрибута), `confidence` и `support`, а также метод `ToString()` для вывода в удобном формате.

Пользовательские метрики

Для задания произвольной метрики пользователь может передать Python-функцию:

Листинг 2: Пример пользовательской метрики (Jaccard на 2-граммах)

```
def jaccard_2gram(a, b):
    def ngrams(s, n=2):
        s = str(s).lower()
        return {s[i:i+n] for i in range(max(1, len(s)-n+1))}
    set_a, set_b = ngrams(a), ngrams(b)
    if not set_a and not set_b:
        return 1.0
    return len(set_a & set_b) / len(set_a | set_b)

algo.set_metrics([jaccard_2gram, equality_metric(), equality_metric()])
```

Такой подход позволяет адаптировать алгоритм к специфике конкретных данных, например, для работы с именами, адресами или текстовыми описаниями, содержащими опечатки и вариации.

Стоит уточнить, что метод `set_metrics()` принимает список метрик сходства, количество которых должно совпадать с числом атрибутов входной таблицы. Каждая метрика применяется к соответствующему столбцу при вычислении попарного сходства значений.

Например, в листинге 2 вызов

```
algo.set_metrics([jaccard_2gram, equality_metric(),
                  equality_metric()])
```

означает, что для первого атрибута используется пользовательская метрика Жаккара на множестве 2-грамм, тогда как для второго и третьего атрибутов применяется стандартная метрика равенства. При построении битовых векторов сходства алгоритм будет сравнивать значения первого столбца с использованием функции `jaccard_2gram`, а значения остальных столбцов — по точному совпадению.

Такой механизм позволяет задавать различные критерии сходства для разных атрибутов таблицы. Например, для текстовых полей могут использоваться метрики строкового сходства (Левенштейн, Жаккар и др.), а для числовых атрибутов — метрики, основанные на абсолютной или относительной разности.

5. Эксперимент

5.1. Исследовательские вопросы

Для оценки эффективности разработанного алгоритма GA-RFD были сформулированы следующие исследовательские вопросы.

- RQ1.** *Какова производительность GA-RFD по сравнению с существующими алгоритмами поиска функциональных зависимостей (Pyro) и приближённых функциональных зависимостей (AFD)?*
- RQ2.** *Способен ли GA-RFD обнаруживать релаксированные функциональные зависимости, которые не выявляются классическими алгоритмами поиска FD и AFD?*
- RQ3.** *Как влияет изменение порога сходства на количество найденных зависимостей и время выполнения алгоритма?*

Для ответа на поставленные вопросы были проведены серии экспериментов на наборах данных различного объёма и структуры.

5.2. Условия эксперимента

Эксперименты проводились на четырёх наборах данных различного объёма и структуры:

- `paper_sample` (7 строк, 3 атрибута) — таблица из оригинальной статьи [1];
- `custom_metric_balls` (10 000 строк, 4 атрибута) — таблица с дробными числовыми атрибутами;
- `neighbors10k` (9 999 строк, 7 атрибутов) — таблица с числовыми атрибутами разных форматов;
- `CIPublicHighway10k` (10 000 строк, 18 атрибутов) — таблица с атрибутами разных типов;

В качестве эталона использовался алгоритм HyFD для поиска точных FD, и Pyro для поиска приближённых FD (AFD) с мерой ошибки g_1 и порогом 0,3.

GA-RFD настраивался в точном режиме (на все атрибуты использовалась метрика равенства, $\text{min_similarity} = 1,0$, $\text{minconf} = 1,0$) и в релаксированном режиме (на все атрибуты использовалась метрика абсолютной разности, $\text{min_similarity} = 0,95$, $\text{minconf} = 0,7$).

Все замеры выполнялись в виртуальной среде Oracle VM VirtualBox на хосте с процессором 13th Gen Intel(R) Core(TM) i5-13420H (6 физических ядер, 12 потоков, базовые такты 2.60 ГГц). Виртуальной машине выделено 7 процессорных ядер и 11 ГБ оперативной памяти. Гостевая ОС — Ubuntu 20.04 (64 bit), компилятор g++ 11.5.0. Для каждого алгоритма измерялось полное время выполнения (без учёта загрузки данных). Для GA-RFD, использующего случайность, проводилось по пять запусков с разными seed; результаты представлены как среднее $\pm$ стандартное отклонение.

5.3. RQ1. Производительность алгоритма

Какова производительность GA-RFD по сравнению с существующими алгоритмами поиска функциональных зависимостей (HyFD) и приближённых функциональных зависимостей (Pyro)?

Для ответа на данный вопрос было проведено сравнение времени выполнения алгоритмов на четырёх наборах данных. Результаты представлены в табл. 5.

Таблица 5: Сравнение времени выполнения (в секундах)

Таблица	HyFD	Pyro	GA-RFD(exact)	GA-RFD(relaxed)
paper_sample	0,009	0,002	0,000 $\pm$ 0,000	0,000 $\pm$ 0,000
custom_metric_balls	0,005	0,002	41,790 $\pm$ 0,266	72,822 $\pm$ 0,122
neighbors10k	0,022	0,007	63,746 $\pm$ 0,306	88,501 $\pm$ 0,223
CIPublicHighway10k	0,033	0,018	166,433 $\pm$ 1,943	845,327 $\pm$ 1,474

Из табл. 5 видно, что существующие алгоритмы существенно превосходят GA-RFD по скорости выполнения на всех исследованных на-

борах данных. Разница особенно заметна на больших таблицах. Так, для набора данных `CIPublicHighway10k` поиск точных функциональных зависимостей занимает у `HyFD` всего 0,033 с, тогда как `GA-RFD` в точном режиме требует в среднем 166,4 с.

Дополнительное увеличение времени наблюдается при переходе к релаксированному режиму. Например, на наборе `CIPublicHighway10k` время выполнения возрастает до 845,3 с, что связано с использованием более сложной метрики сходства и увеличением количества кандидатов, удовлетворяющих условиям отбора.

Ответ на RQ1. `GA-RFD` значительно уступает алгоритмам `HyFD` и `Руго` по скорости выполнения. На исследованных наборах данных замедление составляет от нескольких тысяч до нескольких десятков тысяч раз. Следовательно, алгоритм не предназначен для быстрого поиска точных функциональных зависимостей и должен рассматриваться как средство поиска релаксированных зависимостей, а не как замена существующих алгоритмов.

5.4. RQ2. Обнаружение релаксированных зависимостей

Способен ли `GA-RFD` обнаруживать релаксированные функциональные зависимости, которые не выявляются классическими алгоритмами поиска `FD` и `AFD`?

Для ответа на данный вопрос было выполнено сравнение количества зависимостей, найденных теми же алгоритмами и с теми же параметрами, которые использовались при исследовании производительности в рамках RQ1. В частности, столбец **#FD** содержит количество точных функциональных зависимостей, найденных алгоритмом `HyFD`, а столбец **#AFD** — количество приближённых функциональных зависимостей, найденных алгоритмом `Руго` с использованием меры ошибки g_1 и порога 0,3. Столбцы **#RFD (exact)** и **#RFD (relaxed)** содержат количество зависимостей, найденных алгоритмом `GA-RFD` соответствен-

но в точном и релаксированном режимах. Результаты представлены в табл. 6.

Таблица 6: Сравнение количества найденных зависимостей

Таблица	#FD	#AFD	#RFD (exact)	#RFD (relaxed)
paper_sample	2	6	$3,0 \pm 0,0$	$9,0 \pm 0,0$
custom_metric_balls	12	12	$0,0 \pm 0,0$	$7,0 \pm 0,0$
neighbors10k	12	20	$200,4 \pm 34,2$	$441,0 \pm 0,0$
CIPublicHighway10k	25	79	$391,1 \pm 25,0$	$450,0 \pm 0,0$

Следует отметить, что прямое сравнение количества зависимостей между GA-RFD и Руго является ограничено корректным. Алгоритм Руго возвращает минимальный неизбыточный базис зависимостей, тогда как GA-RFD сохраняет все найденные зависимости, удовлетворяющие заданному порогу уверенности, включая тривиальные и избыточные правила.

Тем не менее результаты позволяют оценить способность алгоритма обнаруживать дополнительные закономерности. На наборе данных `custom_metric_balls` в точном режиме не было найдено ни одной зависимости, тогда как в релаксированном режиме было обнаружено семь правил. Аналогично на наборе `neighbors10k` количество найденных зависимостей увеличилось более чем в два раза.

Полученные результаты свидетельствуют о том, что использование метрик сходства позволяет выявлять закономерности даже в тех случаях, когда строгие функциональные зависимости отсутствуют из-за небольших отклонений значений.

Ответ RQ2. Да, GA-RFD способен обнаруживать релаксированные функциональные зависимости, которые не выявляются классическими алгоритмами поиска FD и AFD. Особенно заметен данный эффект на данных с шумом, погрешностями измерений и незначительными вариациями значений атрибутов.

5.5. RQ3. Влияние порога сходства

Как влияет изменение порога сходства на количество найденных зависимостей и время выполнения алгоритма?

Для оценки влияния релаксации использовалось сравнение точного режима работы алгоритма (`min_similarity=1.0`) и релаксированного режима (`min_similarity=0.95`) в табл. 5. и 6.

На наборе данных `neighbors10k` количество найденных зависимостей увеличилось с 200,4 до 441,0. Для набора `CIPublicHighway10k` число найденных зависимостей возросло с 391,1 до 450,0. Аналогичный эффект наблюдается и на наборе `custom_metric_balls`, где релаксация позволила обнаружить зависимости, полностью отсутствовавшие в точном режиме.

Однако увеличение числа найденных правил сопровождается ростом времени выполнения. Для `CIPublicHighway10k` время работы увеличилось примерно в пять раз, а для `neighbors10k` — почти на 39 %.

Таким образом, снижение порога сходства расширяет пространство поиска и позволяет находить дополнительные закономерности, но приводит к существенному увеличению вычислительных затрат.

Ответ RQ3. Снижение порога сходства приводит к увеличению числа обнаруживаемых зависимостей и позволяет выявлять скрытые закономерности, отсутствующие при строгом сравнении значений. Однако данный эффект достигается ценой заметного роста времени выполнения алгоритма.

5.6. Итоговые выводы

Проведённые эксперименты показали, что разработанный алгоритм GA-RFD успешно решает задачу поиска релаксированных функциональных зависимостей и способен обнаруживать закономерности, недоступные классическим алгоритмам поиска НуFD и Руго. При этом преимуществом алгоритма является гибкость используемых метрик сходства, тогда как основным ограничением остаётся относительно низкая

производительность на больших наборах данных. Следовательно, GARFD целесообразно использовать в задачах анализа данных с шумом, опечатками и неточными соответствиями значений, где полнота обнаружения зависимостей важнее скорости вычислений.

Заключение

В процессе работы были достигнуты следующие результаты:

- Алгоритм GA-RFD реализован на C++ в ядре Desbordante.
- Разработан набор модульных тестов с помощью фреймворка GoogleTest на C++, проверяющих функциональность, корректность работы с различными метриками схожести, краевые случаи (пустые таблицы, один атрибут, одна строка), детерминизм при фиксированном seed, а также корректность вычисления поддержки и работы кэша.
- Выполнена интеграция с Python посредством pybind11.
- Проведён сравнительный эксперимент, показавший, что GA-RFD уступает оптимизированным FD/AFD-алгоритмам (HyFD/Pyro) по скорости в среднем в 3000–5000 раз на таблицах с числом строк порядка 10 000, однако позволяет обнаруживать ослабленные зависимости, не выявляемые классическими методами.

Результаты работы доступны в проекте Desbordante. Исходный код и тесты опубликованы в репозитории на GitHub¹. Реализованный инструмент расширяет возможности профилирования данных за счёт поиска правил, устойчивых одновременно к опечаткам и исключениям.

¹<https://github.com/Desbordante/desbordante-core/pull/750>

Список литературы

- [1] Caruccio Loredana, Deufemia Vincenzo, Polese Giuseppe. A genetic algorithm to discover relaxed functional dependencies from data // [Information Systems](#). — 2020. — Vol. 89. — P. 101482.
- [2] A Catalog of Data Errors / Divya Bhadauria, Hazar Harmouch, Felix Naumann et al. // [ACM Computing Surveys](#). — 2026. — To appear. arXiv preprint. [2604.09277](#).
- [3] [Desbordante: from benchmarking suite to high-performance science-intensive data profiler](#) / Maxim Strutovskiy, Nikita Bobrov, Kirill Smirnov, George Chernishev // [Proceedings of the 8th Joint International Conference on Data Science and Management of Data \(12th ACM IKDD CODS and 30th COMAD\)](#). — ACM, 2024. — P. 234–243.
- [4] Discovering Functional Dependencies through Hitting Set Enumeration / Tobias Bleifuß, Leon Bornemann, Dmitry V. Kalashnikov et al. // [Proc. ACM Manag. Data](#). — 2024. — Vol. 2, no. 1. — P. 1–26.
- [5] Efficient Discovery of Matching Dependencies / Philipp Schirmer, Thorsten Papenbrock, Ioannis Koumarelas, Felix Naumann // [ACM Transactions on Database Systems](#). — 2020. — Aug.. — Vol. 45, no. 3. — P. 1–33. — URL: <https://doi.org/10.1145/3392778>.
- [6] HPI Information Systems Group. Metanome: Data Profiling Platform. — 2024. — Online; accessed: 2024-08-30. URL: <https://hpi.de/naumann/projects/data-profiling-and-analytics/metanome-data-profiling.html>.
- [7] Kivinen Jyrki, Mannila Heikki. Approximate inference of functional dependencies from relations // [Theoretical Computer Science](#). — 1995. — Vol. 149, no. 1. — P. 129–149. — Fourth International Conference on Database Theory (ICDT '92). URL: [https://doi.org/10.1016/0304-3975\(95\)00028-U](https://doi.org/10.1016/0304-3975(95)00028-U).

- [8] Kruse Sebastian, Naumann Felix. Efficient Discovery of Approximate Dependencies // [Proc. VLDB Endow.](#) — 2018. — Vol. 11, no. 7. — P. 759–772.
- [9] [Measuring Approximate Functional Dependencies: a Comparative Study](#) / Marcel Parciak, Sebastiaan Weytjens, Niel Hens et al. // 2024 IEEE 40th International Conference on Data Engineering (ICDE). — IEEE, 2024. — P. 3505–3518. — URL: <https://doi.org/10.1109/ICDE60146.2024.00270>.
- [10] Pandas Development Team. pandas: Python Data Analysis Library. — 2024. — Online; accessed: 2024-08-30. URL: <https://pandas.pydata.org/docs/>.
- [11] Pandas-Profiling Contributors. pandas-profiling: Generate profile report from pandas DataFrame. — 2024. — Online; accessed: 2024-08-30. URL: <https://pypi.org/project/pandas-profiling/>.
- [12] Papenbrock Thorsten, Naumann Felix. [A Hybrid Approach to Functional Dependency Discovery](#) // Proceedings of the 2016 International Conference on Management of Data (SIGMOD '16). — ACM, 2016. — P. 821–833. — URL: <https://doi.org/10.1145/2882903.2915203>.
- [13] TANE: An Efficient Algorithm for Discovering Functional and Approximate Dependencies / Ykä Huhtala, Juha Kärkkäinen, Pasi Porkka, Hannu Toivonen // [The Computer Journal](#). — 1999. — Vol. 42, no. 2. — P. 100–111.
- [14] AFD Metric Calculator and PLIWS : Rep. / Desbordante ; Executor: Максим Емельянов : 2025. — Технический отчёт. URL: <https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/AFDMetricCalculator%20And%20PLIWS%20-%20Emelyanov%20Maksim%20-%202025%20spring.pdf>.
- [15] HyMD Initial Implementation : Rep. / Desbordante ; Executor: Алексей Шлёнских : 2023. — Технический отчёт. URL:

<https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/HyMD%20Initial%20Implementation%20-%20Shlyonskikh%20Alexey%20-%202023%20autumn.pdf>.